DSA 8020 R Lab 6: Non-parametric Regression and Shrinkage Methods

saransh rakshak

Contents

Non-parametric Regression

The dataset `teengamb` concerns a study of teenage gambling in Britain. Type ?teengamb to get more details about the the dataset. In this lab, we will take the variables `gamble` as the response and `income` as the predictor.

Data Source: Ide-Smith & Lea, 1988, *Journal of Gambling Behavior*, 4, 110-118

---

1. Make a scatterplot to examine the relationship between the predictor `income` and the response `gamble`.

Code:

```r
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
library(ggplot2)
library(faraway)
library(splines)
library(mgcv)
```

```
## Loading required package: nlme

##
## Attaching package: 'nlme'

## The following object is masked from 'package:dplyr':
##
##     collapse

## This is mgcv 1.9-1. For overview type 'help("mgcv-package")'.
```

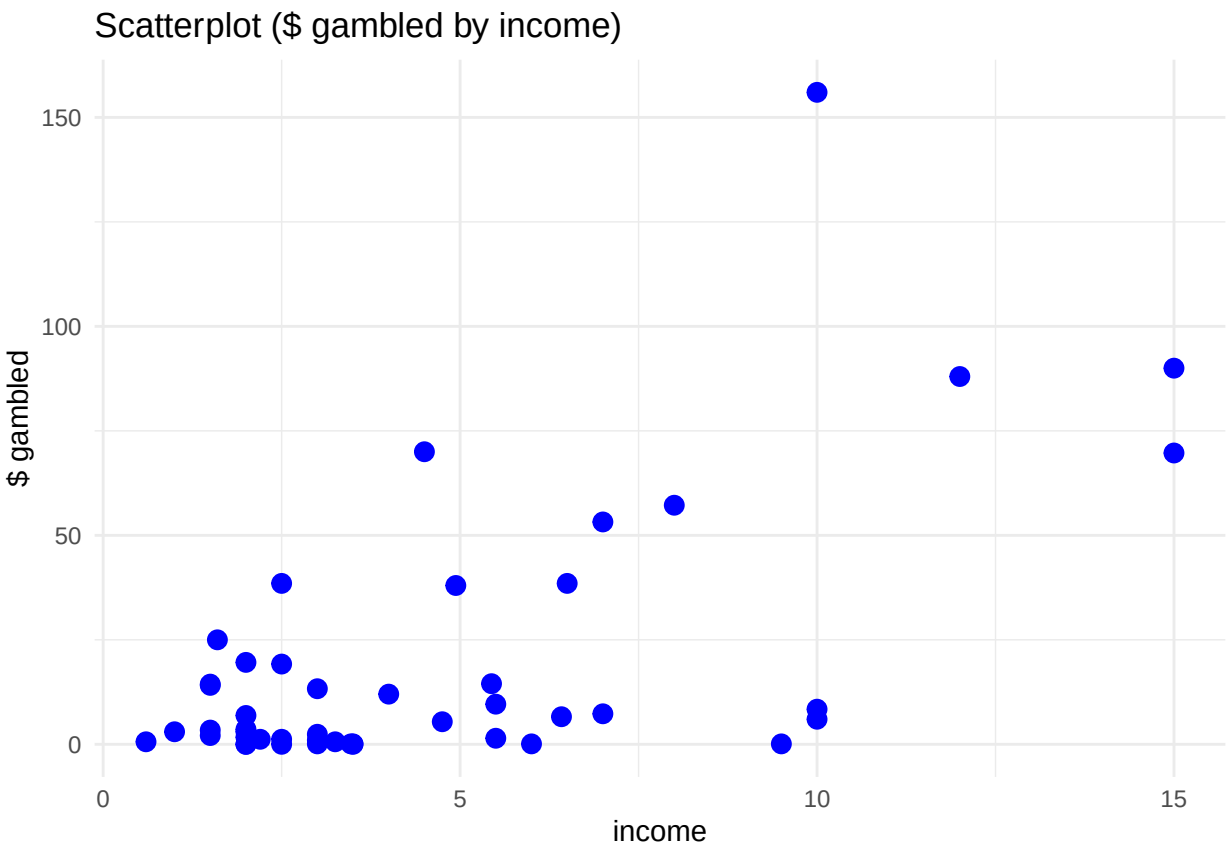
```r
library(glmnet)
```

```
## Loading required package: Matrix

## Loaded glmnet 4.1-8
```

```r
data(teengamb)
head(teengamb)
```

```
##   sex status income verbal gamble
## 1   1     51   2.00      8    0.0
## 2   1     28   2.50      8    0.0
## 3   1     37   2.00      6    0.0
## 4   1     28   7.00      4    7.3
## 5   1     65   2.00      8   19.6
## 6   1     61   3.47      6    0.1
```

```r
ggplot(teengamb, aes(x = income, y = gamble)) +
  geom_point(color = "blue", size = 3) +
  labs(title = "Scatterplot ($ gambled by income)", x = "income", y = "$ gambled") +
  theme_minimal()
```

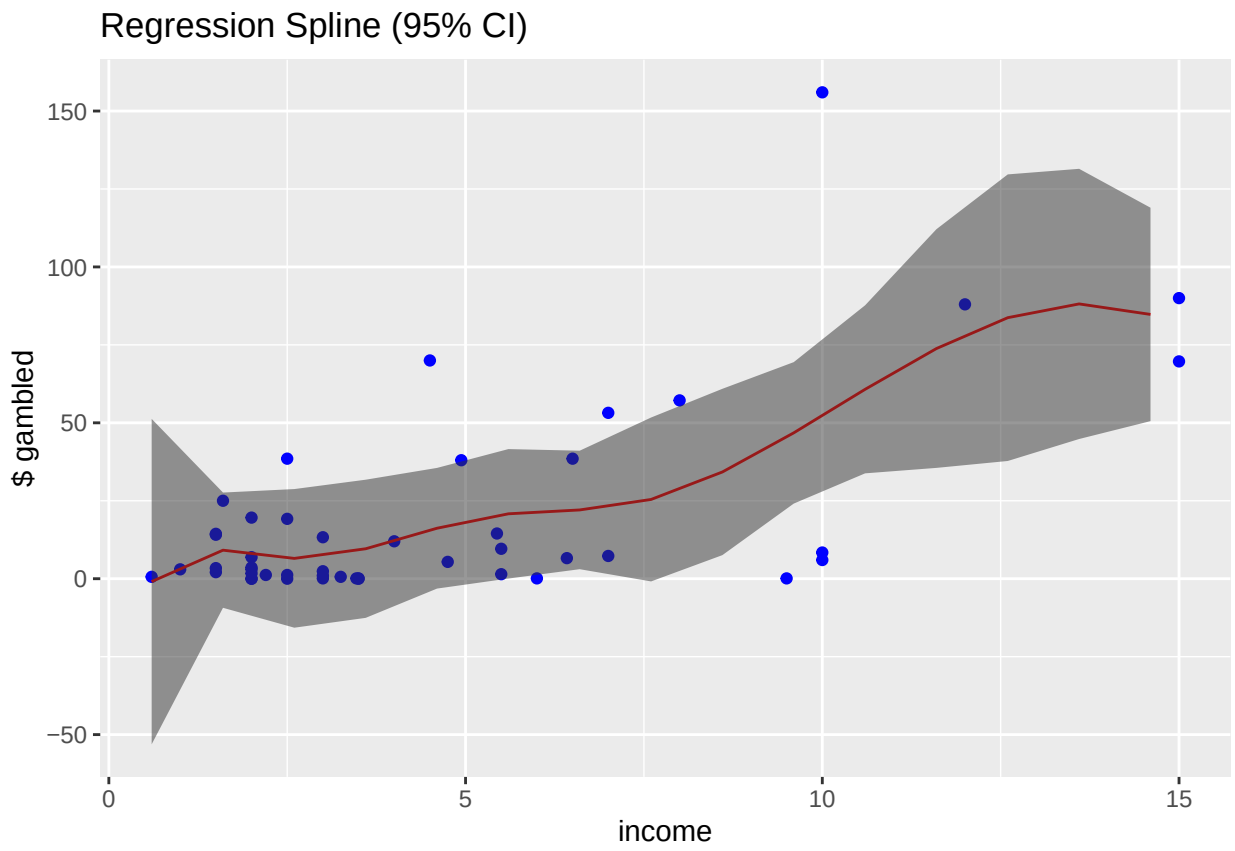


---

2. Fit a curve to the data using a regression spline with `df = 8`. Produce a plot for the fit and a 95% confidence band (using *RegSplinePred <- predict(RegSplineFit, data.frame(income = xg), interval = "confidence")*) for that fit. Is a linear fit plausible?

Code:

```
# regression spline w/ df = 8
RegSplineFit <- lm(gamble ~ bs(income, df = 8), data = teengamb)

# sequence income
xg <- seq(min(teengamb$income), max(teengamb$income))

# 95% CI
RegSplinePred <- predict(RegSplineFit, newdata = data.frame(income = xg), interval = "confidence")
data_plt <- data.frame(income = xg, fit = RegSplinePred[, "fit"], lower = RegSplinePred[, "lwr"], upper

ggplot() +
  geom_point(data = teengamb, aes(x = income, y = gamble), color = "blue") +
  geom_line(data = data_plt, aes(x = income, y = fit), color = "red") +
  geom_ribbon(data = data_plt, aes(x = income, ymin = lower, ymax = upper), alpha = 0.5) +
  labs(title = "Regression Spline (95% CI)", x = "income", y = "$ gambled")
```



Answer:

- Linear fit not possible, data shows non-linear pattern along with 95% CI reflecting the same.

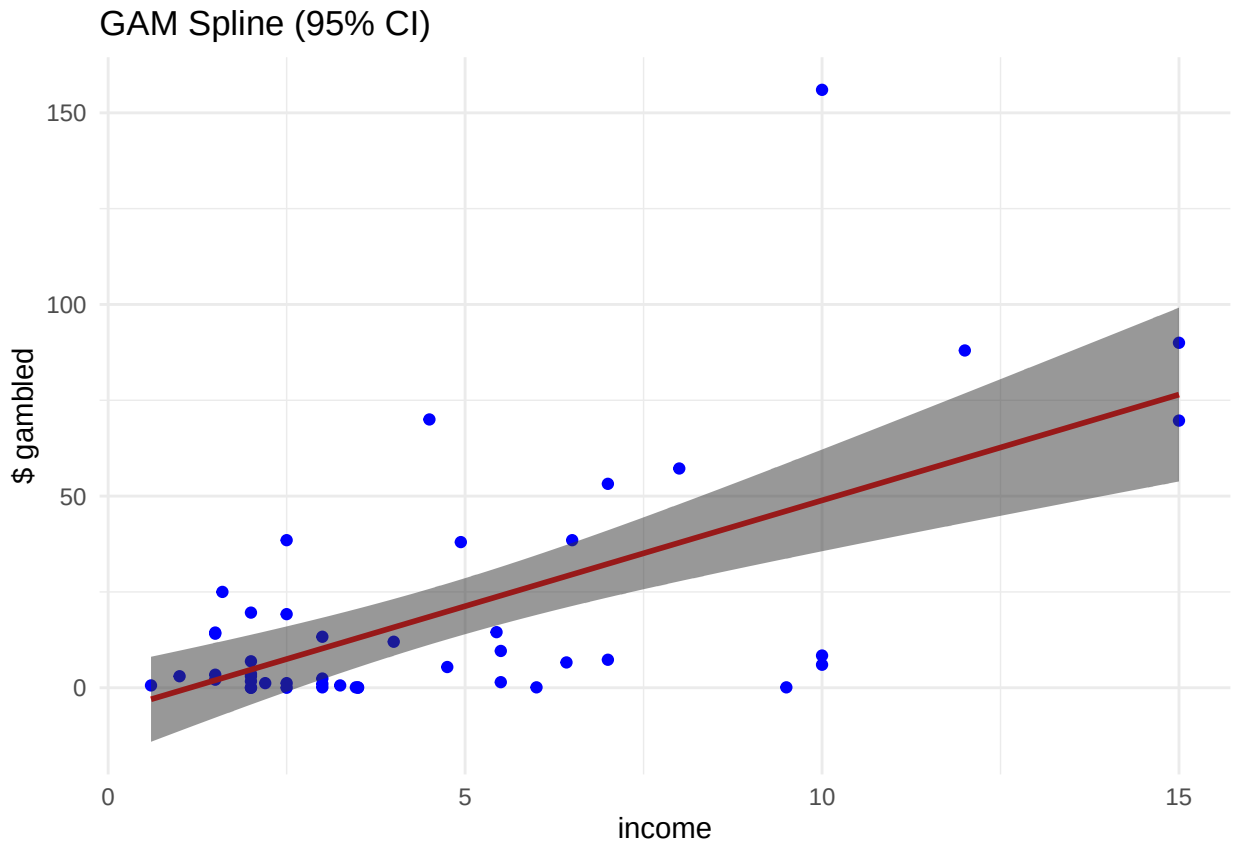
---

3. Fit regression curves using *generalized additive models* and *smoothing splines*, respectively, and compare them with the regression spline fit in problem 2.

Code:

```
gam_fit <- gam(gamble ~ s(income), data = teengamb)

# all.knot to remove dupes
smooth_spline_fit <- smooth.spline(teengamb$income, teengamb$gamble, all.knots = TRUE)

# sequence income
xg <- seq(min(teengamb$income), max(teengamb$income), length.out = 200)

# 95% CI
GAM_Pred <- predict(gam_fit, newdata = data.frame(income = xg), se.fit = TRUE)
data_plt <- data.frame(income = xg, fit = GAM_Pred$fit, lower = GAM_Pred$fit - 2 * GAM_Pred$se.fit,
                       upper = GAM_Pred$fit + 2 * GAM_Pred$se.fit)

# Plot with 95% CI
```

```r
ggplot() +
  geom_point(data = teengamb, aes(x = income, y = gamble), color = "blue") +
  geom_line(data = data_plt, aes(x = income, y = fit), color = "red", linewidth = 1) +
  geom_ribbon(data = data_plt, aes(x = income, ymin = lower, ymax = upper), alpha = 0.5) +
  labs(title = "GAM Spline (95% CI)", x = "income", y = "$ gambled") +
  theme_minimal()
```



Answer:

- GAM plot shows that 'income' affects 'gamble' differently across income range.
- Gambling increases more rapidly after a certain income level.

Ridge Regression andd LASSO: Meat spectrometry to determine fat content

A Tecator Infratec Food and Feed Analyzer, operating in the wavelength range 850 - 1050 nm based on the Near Infrared Transmission (NIT) principle, was employed to collect data on samples of finely chopped pure meat. A total of 215 samples were measured, with both the fat content and a 100-channel spectrum of absorbances recorded for each sample. Due to the time-consuming nature of determining fat content through analytical chemistry, our objective is to construct a model for predicting the fat content of new samples using the 100 absorbances, which can be measured more easily.

Data Source: H. H. Thodberg (1993) "Ace of Bayes: Application of Neural Networks With Pruning", report no. 1132E, Maglegaardvej 2, DK-4000 Roskilde, Danmark

Load the data and partition it into the *training set* (the first 150 observations) and the *testing set* (the remaining 65 observations).

Code:

```
data(meatspec, package = "faraway")
train <- 1:150
test <- 151:215
trainmeat <- meatspec[train,]
testmeat <- meatspec[test,]
```

---

4. Fit a linear regression with all the 100 predictors to the training set. Compute the root mean square error (RMSE) for the testing set. The code below shows how to compute the RMSE for the training set (also known as in-sample prediction), and you will need to modify the code to compute the RMSE for the testing set.

Code:

```
lmFit <- lm(fat ~ ., data = trainmeat)
# Define a function to calculate RMSE
rmse <- function(pred, obs) sqrt(mean((pred - obs)^2))
# Computing RMSE for the training set
rmse(fitted(lmFit), trainmeat$fat)
```

```
## [1] 0.5919489
```

```
# RMSE for test set
rmse(predict(lmFit, newdata = testmeat), testmeat$fat)
```

```
## [1] 3.522791
```

Answer:

- Our RMSE for test data is significantly higher, which may be due to...
  - Overfitting
  - Data variability

---

5. Fit a ridge regression (using cross-validation to select the 'best' λ) to the training set and compute the RMSE for the testing set.

Code:

```r
# fat is reponse var, only include in y
x_train <- as.matrix(trainmeat[, -1])
y_train <- trainmeat$fat
x_test <- as.matrix(testmeat[, -1])
y_test <- testmeat$fat

# cross validation with alpha = 0 for ridge regression
cv_ridge <- cv.glmnet(x_train, y_train, alpha = 0, standardize = TRUE)

# getting best lambda
best_lambda <- cv_ridge$lambda.min
print(c("Best Lambda: ", best_lambda))
```

```
## [1] "Best Lambda: "   "1.2801987849115"
```

```r
# predict on training
train_pred <- predict(cv_ridge, s = best_lambda, newx = x_train)
train_rmse <- rmse(train_pred, y_train)
print(c("Ridge Regression Training RMSE: ", train_rmse))
```

```
## [1] "Ridge Regression Training RMSE: " "1.31062698355935"
```

```r
# pred on test set
test_pred <- predict(cv_ridge, s = best_lambda, newx = x_test)
test_rmse <- rmse(test_pred, y_test)
print(c("Ridge Reg. Test RMSE: ", test_rmse))
```

```
## [1] "Ridge Reg. Test RMSE: " "1.40013893514558"
```

Answer:

- Ridge Training RMSE (1.31)
  - Increased from 0.592 in our linear regression.
  - New ridge regression has higher RMSE because it forfits small amout of training accuracy for increased generalization to data, which allows our test RMSE to increase.
- Ridge Test RMSE (1.4)
  - Decreased (improvement) from 3.52 in our linear regression.
  - Ridge regression improved predition performance on new (test) data by reducing variance with a small increase in bias.

---

6. Fit a LASSO (again using Cross-Validation to select the 'best' λ) to the and training set and compute RMSE for the test set.

Code:

```r
# now using alpha = 1 for lasso regression
cv_lasso <- cv.glmnet(x_train, y_train, alpha = 1, standardize = TRUE)

# getting new best lambda
best_lambda <- cv_lasso$lambda.min
print(c("Best Lambda: ", best_lambda))
```

```
## [1] "Best Lambda: "     "0.373184738898135"
```

```r
# predict on training
train_pred <- predict(cv_lasso, s = best_lambda, newx = x_train)
train_rmse <- rmse(train_pred, y_train)
print(c("Lasso Training RMSE: ", train_rmse))
```

```
## [1] "Lasso Training RMSE: " "0.373184738898132"
```

```r
# pred on test set
test_pred <- predict(cv_lasso, s = best_lambda, newx = x_test)
test_rmse <- rmse(test_pred, y_test)
print(c("Lasso Test RMSE: ", test_rmse))
```

```
## [1] "Lasso Test RMSE: " "0.364903367083932"
```

Answer:

- Lasso Training RMSE (0.37) & Test RMSE (0.36)
  - Considerable improvement over both our ridge regression and linear regression models.
  - Likely due to Lasso selecting only most importance predictors, reduces noise and improves training RMSE.
  - Best balance of bias and variance by removing poor predictors.
  - Improved generalization without sacrificing much training accuracy.

---

7. Fit a LASSO with all the data points (using the best λ from question 6) and report the number of non-zero regression coefficients.

Code:

```r
x_full <- as.matrix(meatspec[, !(names(meatspec) %in% "fat")])
y_full <- meatspec$fat

lasso_full <- glmnet(x_full, y_full, alpha = 1, lambda = best_lambda, standardize = TRUE)

# get coeff
lasso_coefs <- coef(lasso_full)
non_zero_coeff <- sum(lasso_coefs[-1, , drop = FALSE] != 0)  # -1 to remove predictor
selected_preds <- rownames(lasso_coefs)[which(lasso_coefs[-1, , drop = FALSE] != 0) + 1]

print(c("# of non-zero coefficients: ", non_zero_coeff))
```

```
## [1] "# of non-zero coefficients: " "6"
```

```
print(c("non-zero coefficients: ", selected_preds))
```

```
## [1] "non-zero coefficients: " "V7"
## [3] "V8"                      "V9"
## [5] "V40"                     "V41"
## [7] "V63"
```

Answer:

- We can find 6 non-zero coefficients using Lasso method with our optimal lambda = 0.37.
